

Ejercicio Hola Mundo con JPA



Hola Mundo con JPA y Jakarta EE

Explicaremos cada paso y la configuración para que comprendas **por qué se utiliza cada elemento**.

1 Detener GlassFish (si está en ejecución)

Antes de iniciar, debemos detener el servidor de GlassFish para evitar conflictos:

1. Ir a `Services → Servers`
2. Hacer clic derecho sobre `GlassFish Server`
3. Seleccionar `Stop` (si está iniciado)

Esto es necesario para asegurarnos de que el servidor no interfiera con nuestra configuración de JPA.

2 Crear un Proyecto Maven en NetBeans

1. Ir a `File → New Project`

<https://www.globalmentoring.com.mx>

2. Seleccionar Java with Maven → Java Application
3. Click en Next
4. **Configuración del proyecto:**
 - **Project Name:** holamundo-jpa
 - **Ubicación:** C:\Cursos\Java\JakartaEE
 - **Group Id:** sga
 - **Version:** 1.0
 - **Paquete:** sga
5. Click en Finish

✖ **Nota:** Una vez creado, renombramos el proyecto a holamundo-jpa en **Properties** y seleccionamos la opción de Build -> Compile -> Compile on Save

3 Configurar el pom.xml

✖ **Ubicación:** holamundo-jpa/pom.xml

✖ **Código:**

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>sga</groupId>
  <artifactId>holamundo-jpa</artifactId>
  <version>1.0</version>
  <packaging>jar</packaging>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.release>21</maven.compiler.release>
    <exec.mainClass>sga.HolamundoJpa</exec.mainClass>
    <jakartaee>11.0.0-M1</jakartaee>
  </properties>
  <dependencies>
    <dependency>
      <groupId>jakarta.platform</groupId>
      <artifactId>jakarta.jakartaee-api</artifactId>
      <version>${jakartaee}</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>jakarta.persistence</groupId>
      <artifactId>jakarta.persistence-api</artifactId>
      <version>3.2.0</version>
    </dependency>
    <dependency>
      <groupId>org.eclipse.persistence</groupId>
      <artifactId>eclipselink</artifactId>
      <version>4.0.5</version>
    </dependency>
  </dependencies>
</project>
```

```

    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>9.2.0</version>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-api</artifactId>
      <version>2.0.16</version>
      <type>jar</type>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
      <version>2.0.16</version>
      <scope>runtime</scope>
    </dependency>
  </dependencies>
</project>

```

✦ Explicación:

- Se usa **Jakarta EE 11** para compatibilidad con GlassFish.
- **Jakarta Persistence (JPA 3.2.0)** define la API de persistencia.
- **EclipseLink** es la implementación de JPA.
- **MySQL Connector** permite conectarnos a la base de datos.
- **SLF4J** maneja logs para monitorear las operaciones de JPA.

4 Crear la Clase de Entidad `Persona.java`

✦ **Ubicación:** `sga.domain.Persona.java`

✦ **Código:**

```

package sga.domain;

import java.io.Serializable;
import jakarta.persistence.*;

@Entity
public class Persona implements Serializable{
    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    @Column(name="id_persona")
    private int idPersona;

    private String nombre;

    private String apellido;

```

```
private String email;

private String telefono;

public Persona() {
}

public Persona(String nombre, String apellido, String email, String telefono) {
    this.nombre = nombre;
    this.apellido = apellido;
    this.email = email;
    this.telefono = telefono;
}

public int getIdPersona() {
    return idPersona;
}

public void setIdPersona(int idPersona) {
    this.idPersona = idPersona;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getTelefono() {
    return telefono;
}

public void setTelefono(String telefono) {
    this.telefono = telefono;
}

@Override
public String toString() {
    return "Persona{" + "idPersona=" + idPersona + ", nombre=" + nombre + ",
apellido=" + apellido + ", email=" + email + ", telefono=" + telefono + '}';
}
```

}

✦ Explicación:

- `@Entity` indica que esta clase representa una tabla en la BD.
- `@Id` define el campo clave primaria.
- `@GeneratedValue(strategy = GenerationType.IDENTITY)` configura la generación automática del ID.
- `@Column(name = "id_persona")` personaliza el nombre de la columna.

5 Configurar `persistence.xml`

✦ Ubicación: `src/main/resources/META-INF/persistence.xml`**✦ Código:**

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="3.2"
  xmlns="https://jakarta.ee/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_2.xsd">
  <persistence-unit name="PersonaPU" transaction-type="RESOURCE_LOCAL">
    <class>sga.domain.Persona</class>
    <properties>
      <property name="jakarta.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/test?useSSL=false&useTimezone=true&serverTimezone=UTC" />
      <property name="jakarta.persistence.jdbc.user" value="root" />
      <property name="jakarta.persistence.jdbc.password" value="admin" />
      <property name="jakarta.persistence.jdbc.driver"
value="com.mysql.cj.jdbc.Driver" />
      <property name="eclipselink.logging.level.sql" value="FINE" />
      <property name="eclipselink.logging.parameters" value="true" />
      <property name="eclipselink.logging.timestamp" value="false"/>
    </properties>
  </persistence-unit>
</persistence>
```

✦ Explicación:

- Configura la conexión con MySQL.
- Define `PersonaPU` como unidad de persistencia.
- Se usa `EclipseLink` para manejar las transacciones.

6 Crear `ClientePersona.java`

✚ Ubicación: `sga.cliente.ClientePersona.java`

✚ Código:

```
package sga.cliente;

import jakarta.persistence.*;
import sga.domain.Persona;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class ClientePersona {
    private static final Logger logger = LoggerFactory.getLogger(ClientePersona.class);

    public static void main(String[] args) {
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("PersonaPU");
        try (EntityManager em = emf.createEntityManager()) {
            EntityTransaction tx = em.getTransaction();
            //Inicia la transaccion
            tx.begin();
            //No se debe especificar el ID de la base de datos
            Persona persona1 =
                new Persona("Pedro", "Jaramillo", "pjaramillo@mail.com", "44556677");
            logger.info("Objeto a persistir: " + persona1);
            //System.out.println("Objeto a persistir:" + persona1);
            //Persistimos el objeto
            em.persist(persona1);
            //terminamos la transaccion
            tx.commit();
            logger.info("Objeto persistido: " + persona1);
            //System.out.println("Objeto a persistir:" + persona1);
        }
    }
}
```

✚ Explicación:

- `EntityManager` maneja las operaciones con la BD.
- `persist(persona1)` guarda el objeto en la BD.
- `commit()` finaliza la transacción.

7 Ejecutar la Aplicación

1. Click derecho en `ClientePersona.java` → Run
2. Verificar la salida en consola:

```
[main] INFO sga.cliente.ClientePersona - Objeto persistido:
Persona{idPersona=60, nombre=Pedro, apellido=Jaramillo,
email=pjaramillo@mail.com, telefono=44556677}
```

🎉 ¡Felicidades! Has creado tu primera aplicación con JPA y Jakarta EE! 🚀

El código fuente completo lo puedes descargar en la sección de recursos de este video.

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)